

Національний університет «одеська політехніка»

Міністерство освіти і науки України

Інститут комп'ютерних систем

Кафедра інформаційних систем

Протокол лабораторної роботи №1

з дисципліни: «Обчислювальні методи»

Тема: «Розгортання середовища та порівняльний аналіз мовних моделей»

Виконала:

студентка групи AI-245

Жиган Д. О.

Перевірили:

ас. Баранов О.М.

Мета й очікуваний результат

Після роботи студент повинен:

- запустити мовну модель локально й керувати нею з коду;
- розуміти, що локальна модель і хмарна — це один і той самий інтерфейс, і вміти перемикатися між ними зміною двох рядків;
- вимірювати продуктивність інференсу й пояснювати, з чого складається затримка;
- емпірично показати вплив temperature і top_p на стабільність відповідей.

Завдання 1. Ціна роздумів: міркувальна vs інструктивна модель

У першому завданні було проведено порівняння двох мовних моделей: qwen3:4b (міркувальна модель) та llama3.2:3b (інструктивна модель). Для виконання запитів використовувалася програма Ollama через PowerShell.

Спочатку обом моделям було поставлено однакове фактичне питання: «Столиця України? Відповідай одним словом». Після отримання відповідей було визначено кількість згенерованих токенів, загальний час виконання запиту та швидкість генерації токенів.

Таблиця 1 – Результати порівняння моделей на фактичному питанні

Показник	qwen3:4b	llama3.2:3b
response	Київ	Київ.
eval_count	187	5
Час, с	2,618	1,093
Токенів/с	178,32	193,30

Обидві моделі надали правильну відповідь. Проте модель qwen3:4b використала значно більше токенів через додаткові міркування, тоді як llama3.2:3b відповіла швидше та з меншими витратами ресурсів.

Далі було проведено перевірку здатності моделей розв'язувати логічні задачі. Для цього кожній моделі п'ять разів було поставлено задачу про вартість

ручки та зошита, загальна ціна яких становить 110 грн, а різниця у вартості — 100 грн. Правильна відповідь становить 5 грн.

Таблиця 2 – Результати порівняння моделей під час розв'язання логічної задачі

Показник	qwen3:4b	llama3.2:3b
Правильних із 5	5	1
eval_count	1478,6	156,2
Час, с	8,515	0,7134

Примітка. Під час третього запуску моделі llama3.2:3b виникла помилка генерації, тому цей запуск було зараховано як невдалий.

За результатами перевірки модель qwen3:4b правильно розв'язала задачу в усіх п'яти запусках, тоді як llama3.2:3b надала одну правильну відповідь. Міркувальна модель продемонструвала вищу точність, проте потребувала більше часу та токенів для формування відповідей.

Відповіді на запитання до завдання 1

1. Куди пішли додаткові токени міркувальної моделі, якщо обидві моделі дали однакову відповідь?

Додаткові токени модель qwen3:4b використала для формування внутрішніх міркувань, перевірки інформації та побудови остаточної відповіді. Модель llama3.2:3b одразу сформувала відповідь, тому витратила значно менше токенів.

2. На яких типах задач міркувальні моделі мають перевагу, а коли їх використання невиправдане?

Міркувальні моделі краще підходять для складних математичних, логічних задач і програмування, де необхідно виконати декілька послідовних дій. Для простих фактичних питань їх використання часто недоцільне, оскільки вони витрачають більше часу та ресурсів.

3. Чим відрізняється процес навчання міркувальної моделі від інструктивної?

Інструктивні моделі навчають правильно виконувати запити користувача та формувати зрозумілі відповіді. Міркувальні моделі додатково навчають розв'язувати складні задачі за допомогою проміжних обчислень і перевірки отриманих результатів.

4. Чи означає більша кількість згенерованих токенів, що міркувальна модель розумніша за інструктивну?

Ні, більша кількість токенів не завжди означає вищий рівень інтелекту або точності моделі. Вона лише показує, що модель витратила більше ресурсів на формування відповіді, яка не обов'язково буде правильною.

5. Як пояснити різницю у швидкості генерації токенів між моделями qwen3:4b та llama3.2:3b?

Швидкість генерації залежить від кількості параметрів моделі, особливостей її архітектури та характеристик комп'ютера. У проведеному дослідженні модель llama3.2:3b генерувала приблизно 193,30 токена за секунду, а qwen3:4b - 178,32 токена за секунду.

Висновок за завдання 1

У ході виконання завдання було досліджено особливості роботи міркувальної та інструктивної мовних моделей. Встановлено, що для простих фактичних питань інструктивна модель є швидшою та економнішою за кількістю токенів. Водночас під час розв'язання логічної задачі міркувальна модель показала вищу точність завдяки додатковим обчисленням. Отже, вибір моделі залежить від складності завдання та вимог до швидкості й точності відповіді.

Завдання 2. Вплив system-повідомлення на поведінку моделей

У другому завданні було досліджено вплив системного повідомлення на зміст, обсяг, мову та формат відповідей локальних мовних моделей llama3.2:3b і qwen3:4b.

Для виконання завдання відповідно до варіанта 5 використовувався запит: «Поясни різницю між масивом і зв'язним списком».

Для кожної моделі було використано два системні повідомлення:

«Ти лаконічний технічний асистент».

«Відповідай лише українською, максимум двома реченнями».

Для автоматизації дослідження використовувалася програма hello.py мовою Python. Кожна модель отримувала однаковий запит із двома різними системними повідомленнями. Після виконання програма визначала довжину відповіді, приблизну кількість речень та витрати токенів.

Таблиця 3 – Вплив system повідомлення

Модель	System	Довжина відповіді, символів	Мова відповіді	Кількість речень	Інструкцію виконано?
llama3.2:3b	Лаконічний асистент	1869	Змішана, з помилками	16	Ні
llama3.2:3b	українською, 2 речення	344	Українська з помилками	2	Частково
qwen3:4b	Лаконічний асистент	331	Українська	5	Так
qwen3:4b	українською, 2 речення	282	Українська	2	Так

Під час використання першого системного повідомлення модель llama3.2:3b сформуvala надмірно довгу відповідь, що не відповідало вимозі лаконічності. Крім того, у тексті були помилки у поясненні структур даних та некоректні мовні конструкції.

Після застосування другого системного повідомлення llama3.2:3b скоротила відповідь до двох речень, однак припустилася мовних і змістових помилок.

Зокрема, модель неправильно пояснила доступ до елементів зв'язного списку через індекс.

Модель qwen3:4b продемонструвала краще дотримання обох інструкцій. Під час першого запуску вона надала коротке порівняння структур даних, а під час другого сформуvala відповідь українською мовою з двох речень.

Таблиця 4 – Витрати токенів за різних system-повідомлень

Модель	System	prompt_tokens	completion_tokens	total_tokens
llama3.2:3b	Лаконічний асистент	54	571	625
llama3.2:3b	українською,2 речення	58	97	155
qwen3:4b	Лаконічний асистент	57	2569	2626
qwen3:4b	українською,2 речення	66	1059	1125

За результатами вимірювань встановлено, що обмеження кількості речень зменшило довжину відповідей обох моделей.

При цьому qwen3:4b використала значно більше вихідних токенів, ніж llama3.2:3b, хоча її видимі відповіді були коротшими. Це може бути пов'язано з особливостями внутрішнього генерування токенів міркувальною моделлю.

Проведений експеримент показав, що системні повідомлення суттєво впливають на обсяг і форму відповідей мовних моделей.

Модель llama3.2:3b недостатньо точно виконала вимогу лаконічності та допустила помилки у використанні української мови. Модель qwen3:4b продемонструвала краще дотримання встановлених обмежень і надала змістовніші пояснення.

Отже, чітке формулювання системного повідомлення допомагає керувати поведінкою мовної моделі, але не гарантує повної правильності її відповіді.

Відповіді на контрольні запитання до завдання 2

1. Яка модель точніше дотрималась обмеження «максимум двома реченнями»?

Модель qwen3:4b точніше виконала інструкцію, надавши відповідь українською мовою у двох реченнях. Модель llama3.2:3b також дотрималася обмеження, сформувавши одне речення, проте допустила мовні помилки.

2. Чи були випадки, коли модель відповіла не українською, попри пряму вказівку?

Ні, після прямої вказівки обидві моделі відповіли українською мовою, хоча llama3.2:3b допустила мовні помилки. Без цієї вказівки модель qwen3:4b відповіла англійською.

3. Як змінився prompt_tokens при зміні system-повідомлення і чому?

У моделі llama3.2:3b кількість вхідних токенів збільшилася з 59 до 63, а в qwen3:4b — з 65 до 74. Це пов'язано зі зміною тексту системної інструкції та особливостями розбиття тексту на токени.

4. Чому в qwen3:4b значення completion_tokens значно більше, ніж у llama3.2:3b, навіть при однаковій довжині видимої відповіді?

Модель qwen3:4b використовує додаткові токени для внутрішніх міркувань перед формуванням остаточної відповіді. Через це загальна кількість вихідних токенів значно більша, навіть якщо кінцева відповідь коротка.

Відповіді на запитання до чат-бота

1. Чому моделі по-різному дотримуються інструкцій щодо довжини та мови відповіді?

Моделі відрізняються архітектурою, навчальними даними та способами налаштування для виконання інструкцій. Тому одна модель може точніше дотримуватися заданого формату, а інша - допускати відхилення.

2. Чим system-повідомлення відрізняється від user-повідомлення? Чи має воно більший пріоритет?

System-повідомлення задає загальні правила поведінки моделі, а user-повідомлення містить конкретний запит користувача. Зазвичай системна інструкція має вищий пріоритет, хоча модель не завжди ідеально дотримується цієї ієрархії.

3. Чому міркувальна модель витрачає більше вихідних tokenів, навіть коли відповідь однакової довжини?

Міркувальна модель додатково генерує проміжні міркування, які враховуються у витратах tokenів. Інструктивна модель зазвичай переходить безпосередньо до формування відповіді, тому використовує менше tokenів.

4. Що станеться, якщо передати моделі суперечливі інструкції в system і user?

Модель повинна надавати перевагу системній інструкції, оскільки вона має вищий пріоритет. Проте невеликі моделі іноді можуть порушувати це правило та виконувати суперечливі вимоги користувача.

5. Чому малі мовні моделі часто гірше дотримуються інструкцій щодо мови відповіді, ніж великі?

Малі моделі мають обмежені можливості для обробки складних інструкцій і можуть гірше розрізняти мовні особливості. Через це вони частіше змішують мови, допускають граматичні помилки або неправильно використовують терміни.

Висновок до завдання 2

У ході виконання другого завдання було досліджено вплив системних повідомлень на відповіді мовних моделей llama3.2:3b та qwen3:4b.

Встановлено, що використання конкретних інструкцій щодо мови та кількості речень дозволяє значно скоротити обсяг згенерованого тексту. Найкраще дотримання системних повідомлень продемонструвала модель qwen3:4b.

Також було виявлено, що виконання формальних вимог не завжди означає правильність змісту відповіді. Таким чином, системні повідомлення є важливим інструментом керування мовними моделями, однак результати їхньої роботи потребують додаткової перевірки.

Завдання 3. Локальна vs хмарна модель: єдиний інтерфейс

Під час виконання третього завдання було проведено порівняння локальної моделі llama3.2:3b та хмарної моделі openai/gpt-oss-120b. Для цього було створено файли providers.py та compare.py, за допомогою яких здійснювалося підключення до моделей і надсилання однакового запиту.

Для порівняння було використано питання п'ятого варіанта: «Поясни різницю між масивом і зв'язним списком. Коротко». Під час виконання програми було визначено час формування відповіді, кількість вихідних токенів, довжину тексту, мову відповіді та наявність фактичних помилок.

Таблиця 5 – Порівняння локальної та хмарної мовних моделей

Показник	Локальна	Хмарна
Назва моделі	llama3.2:3b	openai/gpt-oss-120b
Час відповіді, с	3,16	1,20
completion_tokens	177	477
Довжина відповіді, символів	577	1109
Мова відповіді	Змішана, з помилками	Українська
Фактичних помилок у відповіді	2	0

За результатами дослідження встановлено, що хмарна модель openai/gpt-oss-120b сформуvala відповідь за 1,20 с, тоді як локальна модель llama3.2:3b витратила 3,16 с.

Хмарна модель надала більш повну та структуровану відповідь українською мовою, правильно пояснивши особливості масиву та зв'язного списку. Локальна модель допустила неточності у визначеннях і використовувала змішані мовні конструкції.

Отже, у проведеному експерименті хмарна модель продемонструвала вищу швидкість і якість відповіді, незважаючи на використання мережевого підключення.

Відповіді на контрольні запитання

1. Скільки рядків коду довелося змінити, щоб перейти з локальної моделі на хмарну?

Безпосередньо в програмі порівняння не довелося змінювати жодного рядка, оскільки параметри моделей були винесені до файлу `providers.py`. Перемикання здійснювалося шляхом вибору відповідного провайдера.

2. Яка відповідь повніша й точніша за змістом? Наведіть конкретний приклад різниці.

Повнішою та точнішою виявилася відповідь хмарної моделі, оскільки вона правильно описала структуру масиву, зв'язного списку та складність операцій. Наприклад, хмарна модель зазначила, що доступ до елемента масиву за індексом має складність $O(1)$, а до елемента зв'язного списку - $O(n)$.

3. Чи були в локальній моделі фактичні помилки або проблеми з українською мовою?

Так, локальна модель допустила неточності у визначеннях структур даних і використовувала змішані українські та іншомовні слова. Це погіршило зрозумілість і точність відповіді.

4. Хмарна модель відповіла швидше за локальну — за рахунок чого, якщо запит передається через інтернет?

Хмарна модель працює на потужному серверному обладнанні, оптимізованому для швидкої обробки запитів. Завдяки цьому час обчислень може бути значно меншим, ніж на локальному комп'ютері, навіть з урахуванням мережевої затримки.

Відповіді на запитання до чат-бота

1. Чому формат запитів OpenAI став стандартом, який підтримують інші провайдери, зокрема Ollama?

Формат запитів OpenAI набув широкого поширення завдяки простоті використання та великій кількості готових бібліотек. Його підтримка дозволяє розробникам використовувати однаковий код для роботи з різними мовними моделями.

2. Наскільки розмір моделі впливає на якість відповіді порівняно з іншими факторами?

Кількість параметрів впливає на можливості моделі, проте не є єдиним показником її якості. Важливу роль також відіграють навчальні дані, архітектура, методи навчання та налаштування генерації.

3. Чому хмарна модель відповіла швидше за локальну, попри затримку мережі?

Хмарні моделі запускаються на спеціалізованому обладнанні, яке забезпечує високу швидкість обчислень. Тому навіть із затримкою передавання даних загальний час відповіді може бути меншим, ніж у локальній моделі.

4. Які ризики для приватності виникають під час використання хмарних API?

Під час використання хмарних API запити передаються на сторонні сервери, що створює ризики витоку конфіденційної інформації. Для захисту даних

використовують шифрування, обмеження доступу, приховування персональної інформації та політики зберігання даних.

5. Що таке rate limit у безкоштовних тарифах і як розробники обходять це обмеження?

Rate limit - це обмеження кількості запитів або токенів, які можна використати за певний проміжок часу. Для роботи в межах лімітів застосовують кешування, черги запитів, повторні спроби із затримкою та оптимізацію використання API.

Висновок за завданням 3

У ході виконання третього завдання було проведено порівняння локальної та хмарної мовних моделей із використанням єдиного програмного інтерфейсу. Встановлено, що хмарна модель openai/gpt-oss-120b надала повнішу та точнішу відповідь за менший час, ніж локальна llama3.2:3b.

На мою думку, локальні моделі доцільно використовувати для роботи з конфіденційною інформацією, за відсутності інтернету або коли важлива незалежність від зовнішніх сервісів. Хмарні моделі краще підходять для складних завдань, які потребують високої якості відповідей і швидкої обробки даних.

Таким чином, використання єдиного інтерфейсу спрощує перемикання між провайдерами та дозволяє обирати модель відповідно до вимог конкретного завдання.

Завдання 4. Вимірювання швидкості: TTFT і залежність від довжини запиту

У четвертому завданні було проведено дослідження швидкості роботи локальної мовної моделі llama3.2:3b та хмарної моделі openai/gpt-oss-120b. Для виконання завдання використовувалася програма мовою Python, яка надсилала запити до моделей через єдиний програмний інтерфейс.

Метою дослідження було визначення часу до появи першого токена (TTFT), загального часу формування відповіді та швидкості генерації тексту в символах за секунду.

Відповідно до варіанта 5 для проведення експерименту було використано три запити різної довжини:

Короткий запит — «Столиця Єгипту?».

Середній запит — «Поясни в одному абзаці, що таке рекурсія».

Довгий запит — «Напиши інструкцію з 10 пунктів, як почати працювати з Git».

Для проведення вимірювань було створено файл `benchmark.py`. Програма використовувала потокове отримання відповіді, що дозволило визначити час появи першого текстового фрагмента. Перед початком вимірювань виконувався прогрів моделей, після чого кожен запит запускався тричі без паралельного виконання.

Для підвищення точності отриманих результатів обчислювалися медіанні значення трьох запусків. Результати було збережено у файлі `results.csv`.

Таблиця 6 – Результати вимірювання швидкості мовних моделей

Розміщення	Модель	Промпт	TTFT, с	Всього, с	Симв./с
локальна	llama3.2:3b	короткий	2,023	2,086	13,4
локальна	llama3.2:3b	середній	2,022	2,563	122,7
локальна	llama3.2:3b	довгий	2,040	5,000	385,6
хмарна	openai/gpt-oss-120b	короткий	0,274	0,308	73,5
хмарна	openai/gpt-oss-120b	середній	0,528	0,679	514,0
хмарна	openai/gpt-oss-120b	довгий	0,556	2,409	1134,1

За результатами вимірювань встановлено, що хмарна модель `openai/gpt-oss-120b` продемонструвала менший час до появи першого токена та вищу швидкість генерації відповідей для всіх трьох типів запитів.

Для короткого запиту загальний час відповіді локальної моделі становив 2,086 с, тоді як хмарної - 0,308 с. При виконанні довгого запиту час збільшився до 5,000 с та 2,409 с відповідно. Таким чином, хмарна модель виявилася швидшою, хоча відносна різниця за загальним часом виконання зменшилася при переході до довгого запиту.

Додатково було проведено порівняння холодного та теплого старту локальної моделі llama3.2:3b. Для цього модель було зупинено за допомогою Ollama, після чого виконано перший запит і виміряно його тривалість. Наступний запит виконувався без повторної зупинки моделі.

Для вимірювання використовувався файл `start_test.py`, у якому було реалізовано фіксацію часу до появи першого токена та загального часу відповіді.

Таблиця 7 – Холодний і теплий старт локальної моделі

Показник	Холодний старт	Теплий старт
TTFT, с	3,618	2,024
Всього, с	3,679	2,331

За результатами дослідження встановлено, що холодний старт потребує більше часу, ніж теплий. Це пояснюється необхідністю завантаження моделі в оперативну або відеопам'ять під час першого запиту. При повторному зверненні модель уже знаходиться в пам'яті, тому початок генерації відбувається швидше.

Для наочного порівняння результатів було побудовано стовпчикову діаграму засобами бібліотеки Matplotlib. На діаграмі відображено швидкість генерації відповідей у символах за секунду для шести комбінацій моделей та типів запитів.

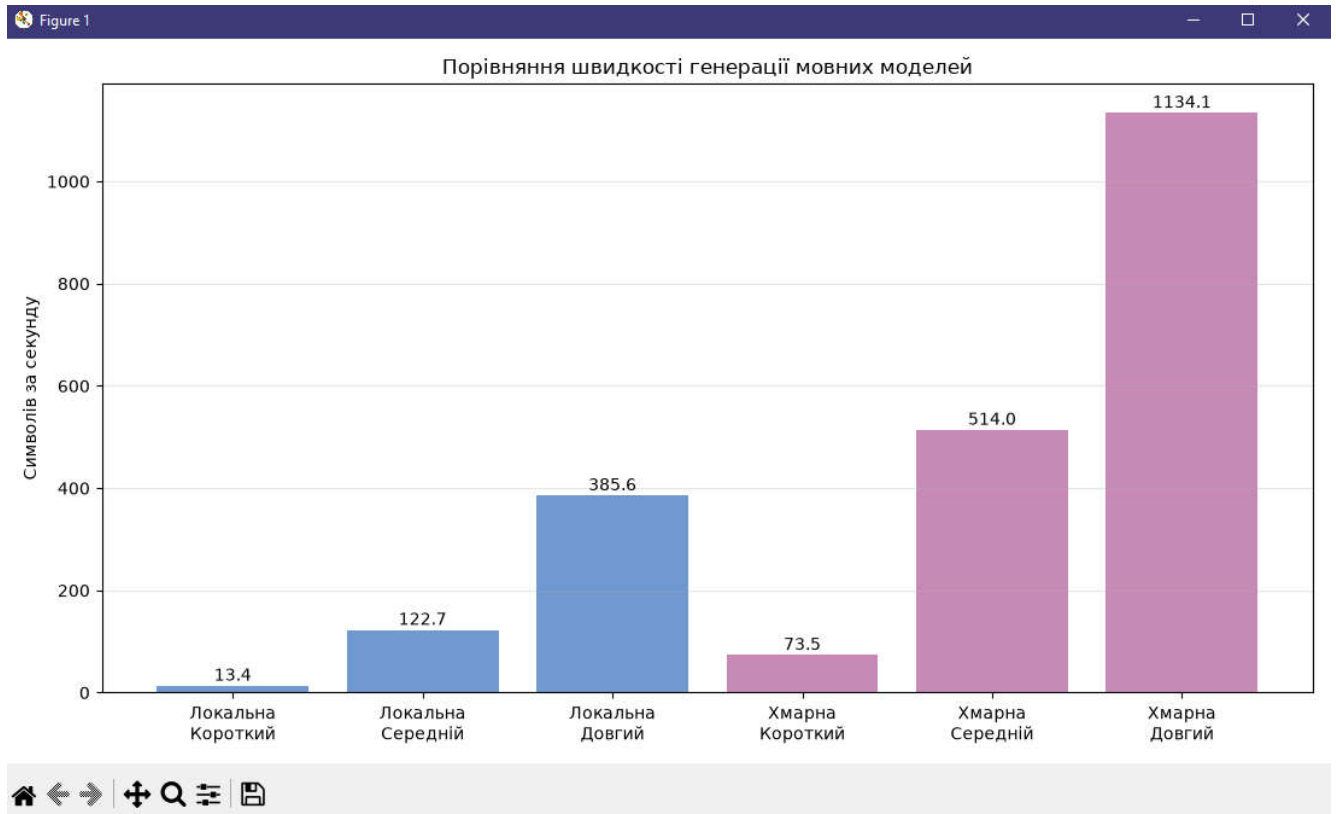


Рисунок 1 – Порівняння швидкості генерації відповідей локальної та хмарної мовних моделей

З отриманої діаграми видно, що хмарна модель забезпечила вищу швидкість генерації тексту для кожного типу запити. Найбільше значення швидкості становило 1134,1 символа за секунду для хмарної моделі під час виконання довгого запити, тоді як локальна модель досягла 385,6 символа за секунду.

Відповіді на контрольні запитання

1. У якої моделі кращий TTFT і чому саме в неї?

Кращий показник TTFT продемонструвала хмарна модель openai/gpt-oss-120b, оскільки для всіх типів запитів час до появи першого токена був меншим. Це пояснюється використанням потужного серверного обладнання та оптимізованих механізмів обробки запитів.

2. Чи збігається «найшвидша за TTFT» з «найшвидшою за символами/с»? Якщо ні - поясніть.

Так, у проведеному дослідженні хмарна модель продемонструвала кращі показники як за TTFT, так і за швидкістю генерації символів. Однак загалом ці показники не завжди збігаються, оскільки час до першого токена та швидкість подальшої генерації залежать від різних факторів.

3. Як змінилося співвідношення локальної та хмарної моделі при переході від короткого промпту до довгого?

Під час виконання короткого запиту локальна модель витратила 2,086 с, а хмарна - 0,308 с. Для довгого запиту час збільшився до 5,000 с та 2,409 с відповідно, тому хмарна модель залишилася швидшою, але її відносна перевага за загальним часом зменшилася.

4. Наскільки різняться три повтори одного виміру? Чи достатньо було б одного запуску?

Результати повторних запусків відрізнялися залежно від навантаження на обладнання, мережових затримок і довжини згенерованої відповіді. Наприклад, під час довгого запиту локальної моделі загальний час змінювався від 3,990 до 5,687 с, тому одного запуску недостатньо для надійного оцінювання продуктивності.

5. Чому холодний старт повільніший за теплий?

Під час холодного старту необхідно завантажити модель у пам'ять комп'ютера та підготувати її до виконання запиту. При теплому старті модель уже знаходиться в пам'яті, тому затримка перед формуванням відповіді зменшується.

Відповіді на запитання до чат-бота

1. Що фізично відбувається під час TTFT — часу до першого токена — і чому цей показник вимірюють окремо від загальної швидкості генерації?

Під час TTFT відбувається передавання запиту, обробка вхідних даних і підготовка моделі до генерації першого токена. Цей показник вимірюють окремо,

оскільки він визначає початкову затримку відповіді, а швидкість генерації характеризує подальше створення тексту.

2. Чому хмарна модель може мати гірший TTFT, але кращу швидкість генерації, ніж локальна?

Хмарна модель може витратити додатковий час на передавання запиту через інтернет та очікування доступу до серверних ресурсів. Водночас потужне серверне обладнання може забезпечувати значно швидшу генерацію тексту після початку відповіді.

3. Чому перший запуск локальної моделі повільніший за наступні і що таке холодний старт?

Холодний старт - це перший запуск моделі після її вивантаження з пам'яті. Він потребує додаткового часу на завантаження параметрів моделі, тоді як наступні запити можуть використовувати вже підготовлену модель.

4. Чому при вимірюванні продуктивності беруть медіану кількох запусків, а не одне значення?

Медіана дозволяє зменшити вплив випадкових затримок та аномально великих або малих значень. Використання декількох запусків забезпечує більш надійне порівняння продуктивності моделей.

5. Чому обробка вхідного запиту йде швидше, ніж генерація відповіді такої ж довжини?

Вхідні токени можуть оброблятися паралельно, що дозволяє ефективно використовувати обчислювальні ресурси. Під час генерації вихідних токенів модель зазвичай створює їх послідовно, тому цей процес потребує більше часу.

Висновок до завдання 4

У ході виконання четвертого завдання було досліджено швидкість роботи локальної та хмарної мовних моделей залежно від довжини запиту. Було визначено час до появи першого токена, загальний час формування відповіді та швидкість генерації символів.

За результатами експерименту встановлено, що хмарна модель `openai/gpt-oss-120b` продемонструвала кращі показники швидкості для короткого, середнього та довгого запитів. Також було підтверджено, що холодний старт локальної моделі повільніший за теплий через необхідність завантаження моделі в пам'ять комп'ютера.

Отримані результати дозволили зробити висновок, що для інтерактивних систем важливо мінімізувати затримку до першого токена, а для обробки великих обсягів інформації = забезпечувати високу швидкість генерації та мінімальний загальний час виконання запиту.

Завдання 5. Оцінювання якості моделей за рубрикою

У п'ятому завданні було проведено порівняльне оцінювання якості відповідей трьох мовних моделей: локальних `Llama3.2:3b` і `qwen3:4b` та хмарної `openai/gpt-oss-120b`.

Метою дослідження було визначення здатності моделей правильно відповідати на фактичні запитання, дотримуватися заданого формату, стискати текст без втрати інформації, генерувати програмний код, виконувати логічні обчислення та формувати грамотні відповіді українською мовою.

Відповідно до варіанта 5 «Фінанси» було підготовлено шість тестових завдань:

1. Фактичні знання — визначити рік введення гривні як національної валюти України.
2. Дотримання формату — сформувати JSON з полями `currency`, `code` та `year`.
3. Стиснення тексту — скоротити текст про історію гривні до двох речень зі збереженням усіх числових значень.

4. Генерація коду — написати функцію `compound_interest(principal, rate, years)` для обчислення складних відсотків.
5. Логічне мислення — визначити кінцеву суму депозиту 20 000 грн під 12% річних з урахуванням податку 19,5% на відсотки.
6. Якість української мови — пояснити відмінність між акцією та облігацією двома абзацами.

Для автоматизації перевірки було створено програму `quality.py` мовою Python. Кожне завдання виконувалося двічі для кожної моделі з параметром `temperature = 0,7`. Загалом було отримано 36 відповідей, які зберігалися у файлі `quality.json`.

Оцінювання проводилося за трибальною шкалою: 0 балів - завдання не виконано; 1 бал - завдання виконано частково; 2 бали - завдання виконано повністю. Максимальна оцінка для однієї моделі становила 12 балів.

Таблиця 8 – Оцінка якості моделей за рубрикою

Завдання	llama3.2:3b	qwen3:4b	Хмарна модель
1. Факт	0	0	2
2. Формат	2	2	2
3. Стиснення	0	2	2
4. Код	2	2	1
5. Логіка	0	2	2
6. Українська	0	1	1
Разом /12	4	9	10

За результатами оцінювання найвищий бал отримала хмарна модель `openai/gpt-oss-120b` — 10 із 12 можливих. Модель `qwen3:4b` набрала 9 балів, а `llama3.2:3b` — 4 бали.

Найбільші відмінності спостерігалися під час перевірки фактичних знань, стиснення тексту та логічного мислення. Модель `llama3.2:3b` допускала значні помилки у фактах, втрачала числові дані та неправильно виконувала обчислення.

У завданні на генерацію коду локальні моделі правильно реалізували необхідну функцію. Хмарна модель також сформулила правильний алгоритм, однак додала Markdown-обгортку, хоча за умовою вимагалось надати лише код.

Для перевірки стабільності моделей було порівняно оцінки, отримані за результатами двох запусків кожного завдання.

Таблиця 9 – Розбіжність оцінок між двома запусками

Завдання	llama3.2:3b	qwen3:4b	Хмарна модель
1. Факт	Ні	Ні	Ні
2. Формат	Ні	Ні	Ні
3. Стиснення	Ні	Ні	Ні
4. Код	Ні	Ні	Ні
5. Логіка	Ні	Ні	Ні
6. Українська	Ні	Ні	Ні

Порівняння показало, що оцінки двох запусків за встановленою рубрикою не відрізнялися. Проте самі відповіді іноді були різними. Наприклад, у фактичному завданні модель llama3.2:3b під час першого запуску вказала 1992 рік, а під час другого - 1961 рік. Обидві відповіді були неправильними, тому отримали однакову оцінку - 0 балів.

Це свідчить про те, що однакові оцінки не завжди означають повну стабільність відповідей мовної моделі.

Приклад провалу мовної моделі

Для детальнішого аналізу було обрано помилку локальної моделі llama3.2:3b у завданні на перевірку фактичних знань.

Модель: llama3.2:3b.

Повний текст запиту:

«У якому році гривню ввели як національну валюту України? Відповідай лише роком.»

Повна відповідь першого запуску:

1992.

Повна відповідь другого запуску:

1961

Правильна відповідь:

1996 рік.

Правильність відповіді перевірено за інформацією Національного банку України щодо історії запровадження гривні.

Пояснення помилки:

Модель двічі надала неправильну відповідь, причому вказала різні роки. Це демонструє недостатню надійність її фактичних знань. Подібне явище називають галюцинацією мовної моделі, коли вона формує переконливу на вигляд відповідь, яка не відповідає дійсності.

Також показовою виявилася задача на логічне мислення, у якій необхідно було розрахувати суму депозиту з урахуванням податку.

Правильний розрахунок:

Нараховані відсотки: $20\,000 \times 0,12 = 2\,400$ грн.

Податок: $2\,400 \times 0,195 = 468$ грн.

Чистий дохід: $2\,400 - 468 = 1\,932$ грн.

Кінцева сума: $20\,000 + 1\,932 = 21\,932$ грн.

Модель llama3.2:3b під час другого запуску отримала неправильний результат - 17 632 грн, тоді як qwen3:4b і openai/gpt-oss-120b правильно визначили кінцеву суму 21 932 грн.

Відповіді на контрольні запитання

1. Яка модель набрала найбільше балів і на яких саме завданнях виникла основна різниця?

Найбільше балів набрала хмарна модель openai/gpt-oss-120b — 10 із 12. Основна різниця між моделями виникла під час перевірки фактичних знань, стиснення тексту та логічного мислення.

2. На яких завданнях оцінки двох прогонів розійшлися? Що це говорить про надійність моделі?

Оцінки двох запусків не відрізнялися за встановленою шкалою. Однак окремі відповіді мали різний зміст, тому однаковий бал не гарантує стабільності та достовірності результатів моделі.

3. Чи виконала хоч одна модель вимогу «без markdown» у завданні 2?

Так, усі три моделі надали відповіді у правильному форматі JSON без Markdown-обгортки. Проте локальні моделі вказали неправильні значення року, що свідчить про різницю між правильністю формату та достовірністю даних.

4. Чи були проблеми з українською мовою — русизми, кальки, перехід на іншу мову?

Так, найбільше мовних помилок було виявлено у відповідях llama3.2:3b. Моделі qwen3:4b і openai/gpt-oss-120b також іноді використовували неприродні конструкції, проте загалом краще дотримувалися української мови.

Відповіді на запитання до чат-бота

1. Які ще підходи до оцінювання якості мовних моделей використовують на практиці?

Крім оцінювання за рубрикою, використовують автоматизовані тести, стандартні набори завдань, експертне оцінювання та порівняння відповідей різних моделей. Також перевіряють фактичну точність, стабільність, логічність і відповідність інструкціям.

2. Чому мовні моделі часто ігнорують пряму інструкцію не використовувати Markdown при генерації JSON?

Під час навчання моделі часто зустрічають приклади програмного коду та JSON, оформлені за допомогою Markdown. Через це модель може автоматично

відтворювати звичний формат навіть тоді, коли користувач просить надати лише JSON.

3. Чому малі мовні моделі гірше пишуть українською, ніж англійською?

Це може бути пов'язано з меншою кількістю україномовних навчальних даних порівняно з англійськими. Також невеликий розмір моделі обмежує її можливості щодо правильного використання складної граматики, лексики та термінології.

4. Чому та сама модель на однаковий запит може дати відповіді різної якості при повторному запуску?

Генерація тексту має ймовірнісний характер, тому модель може обирати різні варіанти наступних токенів. Через це відповіді на однаковий запит іноді відрізняються за змістом, форматом і правильністю.

5. Що таке галюцинація мовної моделі і чому модель радше вигадася факт, ніж визнає незнання?

Галюцинація - це генерування неправдивої або вигаданої інформації, яку модель подає як достовірну. Це відбувається тому, що модель формує найімовірніше продовження тексту, а не обов'язково перевіряє факти за авторитетними джерелами.

Висновок до завдання 5

У ході виконання п'ятого завдання було проведено порівняльне оцінювання якості трьох мовних моделей за шістьма критеріями. Для кожного завдання було виконано два запуски, після чого відповіді перевірялися відповідно до встановленої рубрики.

За результатами дослідження найвищий бал отримала хмарна модель openai/gpt-oss-120b - 10 із 12 можливих. Модель qwen3:4b набрала 9 балів, а llama3.2:3b - 4 бали.

Встановлено, що локальні моделі можуть успішно генерувати програмний код і дотримуватися заданого формату, але водночас допускати помилки у фактичних знаннях та логічних обчисленнях. Також було виявлено недоліки у використанні української мови.

Отже, оцінювання за рубрикою дозволяє визначити сильні та слабкі сторони мовних моделей. Для отримання надійних результатів важливо проводити повторні запуски, перевіряти правильність відповідей і враховувати не лише загальний бал, а й особливості виконання окремих завдань.

Завдання 6. Керування випадковістю: temperature і top_p

У шостому завданні було досліджено вплив параметрів temperature і top_p на різноманітність, правильність та відтворюваність відповідей мовної моделі llama3.2:3b.

Метою дослідження було визначення залежності результатів генерації від налаштувань випадковості та встановлення найбільш придатних значень параметрів для творчих і фактичних завдань.

Для проведення експерименту відповідно до варіанта 5 було використано два запити:

Творче завдання: «Придумай назву застосунку для пошуку партнерів для тренувань. Відповідай лише назвою».

Фактичне запитання: «У якому році гривню ввели як національну валюту України? Відповідай лише роком».

Для автоматизації дослідження було створено програму sampling.py мовою Python. Творчий запит виконувався по п'ять разів для значень temperature 0,0; 0,7; 1,5 та значень top_p 0,1; 0,5; 1,0. Під час перевірки top_p температура залишалася сталою і дорівнювала 0,7.

Фактичне запитання виконувалося по п'ять разів при temperature 0,0 і 1,5. Додатково було виконано десять повторів фактичного запиту при temperature 0,0 для перевірки відтворюваності.

Таблиця 10 – Вплив параметрів на різноманітність відповідей

Параметр	Значення	Унікальних із 5 (творче)	Правильних із 5 (фактичне)
temperature	0.0	1	0
temperature	0.7	5	—
temperature	1.5	5	0
top_p	0.1	1	—
top_p	0.5	3	—
top_p	1.0	5	—

За результатами дослідження встановлено, що при temperature = 0,0 модель п'ять разів згенерувала однакову назву «ТренерPlus». При temperature = 0,7 було отримано п'ять різних назв, які переважно відповідали тематиці застосунку.

При temperature = 1,5 також було отримано п'ять унікальних відповідей, однак їхня якість погіршилася. Зокрема, модель створювала назви зі змішаними мовами та некоректними символами, наприклад «FitNESS-парnar.com» і «Схваśай!».

Під час зміни top_p спостерігалось поступове збільшення різноманітності відповідей. При top_p = 0,1 було отримано одну унікальну назву, при 0,5 - три, а при 1,0 - п'ять.

У фактичному завданні правильна відповідь становить 1996 рік. Проте при temperature = 0,0 модель п'ять разів відповіла «1992», тобто не надала жодної правильної відповіді. При temperature = 1,5 модель називала різні неправильні роки: 1987, 1938, 1948 та 1992.

Ці результати показують, що збільшення температури підвищує випадковість відповідей, але не гарантує їхньої правильності. Низька температура робить генерацію передбачуванішою, проте модель може стабільно повторювати фактичну помилку.

Для перевірки відтворюваності фактичний запит було виконано десять разів із параметром temperature = 0,0.

Таблиця 11 – Перевірка відтворюваності

Показник	Значення
Кількість запусків	10
temperature	0,0
Унікальних відповідей	1
Усі відповіді однакові (так / ні)	Так

Під час усіх десяти запусків модель надала однакову відповідь — «1992». Отже, в цьому експерименті $temperature = 0,0$ забезпечила повну повторюваність тексту відповіді, але не її фактичну правильність.

Відповіді на контрольні запитання

1. Яке значення температури ви поставите агенту, який має повертати JSON, і чому?

Для агента, який повинен повертати JSON, я обрала б $temperature = 0,0$ або близьке до нуля значення. Це зменшує випадковість генерації та допомагає отримувати стабільніші результати, хоча саме по собі не гарантує правильного формату.

2. Яке значення температури ви поставите генератору варіантів заголовків?

Для генератора заголовків я обрала б $temperature = 0,7$, оскільки в експерименті це значення дозволило отримати п'ять різних назв. При $temperature = 1,5$ різноманітність залишилася високою, але з'явилися безглузді та некоректно сформовані слова.

3. Чим вплив top_p відрізняється від впливу $temperature$ за характером, а не за силою?

$Temperature$ змінює співвідношення ймовірностей можливих наступних токенів, роблячи розподіл більш або менш випадковим. Top_p обмежує набір

допустимих токенів, залишаючи лише ті, сукупна ймовірність яких досягає заданого порога.

4. Чи означає $temperature = 0$ повну відтворюваність результату? Що показав ваш експеримент із 10 запусків?

$Temperature = 0$ не гарантує абсолютно однакових результатів за всіх умов. Однак у проведеному експерименті модель десять разів надала однакову відповідь «1992», хоча правильним роком є 1996.

5. Наведіть найбезглуздішу відповідь, яку модель видала при $temperature = 1,5$, і поясніть, чому вона взагалі могла з'явитися.

Однією з найбезглуздіших відповідей стала назва «FitNESS-парnar.com», у якій змішано українські та англійські елементи. Вона могла з'явитися через високу температуру, яка збільшує ймовірність вибору менш передбачуваних токенів.

Відповіді на запитання до чат-бота

1. Що саме математично робить параметр $temperature$ із розподілом ймовірностей наступного токена?

$Temperature$ змінює розподіл імовірностей перед вибором наступного токена шляхом масштабування оцінок можливих токенів. Низька температура збільшує перевагу найімовірніших варіантів, а висока робить розподіл більш рівномірним і підвищує різноманітність відповідей.

2. Чим top_p відрізняється від $temperature$ — обидва впливають на різноманітність, але за яким механізмом?

$Temperature$ впливає на співвідношення ймовірностей між можливими токенами. Top_p відбирає найімовірніші токени, сукупна ймовірність яких досягає заданого порога, та виключає менш імовірні варіанти з подальшого вибору.

3. Чи є сенс змінювати обидва параметри одночасно, чи краще налаштовувати щось одне?

Змінювати обидва параметри одночасно можна, оскільки вони керують різними аспектами генерації. Однак під час експериментів доцільніше спочатку змінювати один параметр, залишаючи інший сталим, щоб можна було визначити вплив кожного з них.

4. Чому навіть при $temperature = 0$ мовна модель не гарантує абсолютно однакову відповідь при повторних запусках?

Навіть при $temperature = 0$ на результат можуть впливати особливості паралельних обчислень, числові похибки та зміни середовища виконання. Тому низька температура суттєво зменшує випадковість, але не забезпечує абсолютної гарантії однакового результату.

5. Які значення $temperature$ використовують на практиці для витягу структурованих даних, а які для генерації творчого тексту?

Для витягу структурованих даних зазвичай використовують низькі значення температури, наприклад від 0,0 до 0,2, щоб зменшити варіативність відповідей. Для творчих завдань частіше застосовують вищі значення, наприклад від 0,7 до 1,0, оскільки вони дозволяють генерувати різноманітніші ідеї.

Власні спостереження

На мою думку, для наступних лабораторних робіт, у яких модель повинна повертати структуровані дані, найдоцільніше використовувати $temperature = 0,0$ та обмежувати `top_p`, якщо потрібно додатково зменшити різноманітність.

Дослідження показало, що параметри за замовчуванням не завжди підходять для конкретного завдання. Наприклад, для творчої генерації $temperature = 0,0$ виявилася занадто обмежувальною, тоді як для фактичних запитань висока температура призводила до появи різних неправильних відповідей.

Водночас низька температура не забезпечує достовірності інформації. Модель може впевнено та багаторазово повторювати одну й ту саму помилку, тому фактичні відповіді необхідно додатково перевіряти.

Висновок до завдання 6

У ході виконання шостого завдання було досліджено вплив параметрів temperature і top_p на різноманітність, правильність та відтворюваність відповідей локальної мовної моделі llama3.2:3b.

Встановлено, що зі збільшенням temperature і top_p зростала кількість унікальних відповідей на творче завдання. При temperature = 0,7 було отримано п'ять різних назв застосунку, тоді як при temperature = 1,5 з'явилися некоректні мовні конструкції.

У фактичному завданні модель не надала жодної правильної відповіді ні при temperature = 0,0, ні при temperature = 1,5. Перевірка відтворюваності показала, що за десять запусків із temperature = 0,0 було отримано лише одну унікальну відповідь, яка водночас була неправильною.

Отже, temperature і top_p дозволяють керувати випадковістю та різноманітністю генерації, але не гарантують точності інформації. Для структурованих і фактичних завдань доцільно використовувати низьку температуру, а для творчих - помірно підвищену.

Загальні висновки

Таблиця 12 – Підсумкове порівняння моделей

Критерій	Локальна мала (3B)	Локальна більша (4B/8B)	Хмарна
Швидкість генерації	13,4–385,6 симв./с	Не вимірювалася в завданні 4	73,5–1134,1 симв./с
TTFT	2,022–2,040 с	Не вимірювався в завданні 4	0,274–0,556 с
Якість, балів /12	4	9	10

Стабільність формату (завдання 2)	JSON правильний за синтаксисом, але з помилками в даних	JSON правильний за синтаксисом, але з помилками в даних	JSON правильний за синтаксисом і змістом
Потрібні ресурси	Ресурси власного комп'ютера, порівняно невеликі вимоги	Ресурси власного комп'ютера, більші вимоги, ніж у 3B	Серверні ресурси провайдера
Вартість	Без окремої оплати запитів, витрати на обладнання та електроенергію	Без окремої оплати запитів, витрати на обладнання та електроенергію	Залежить від тарифів і лімітів API
Куди йдуть ваші дані	Обробляються локально	Обробляються локально	Передаються зовнішньому провайдеру через інтернет
Працює без інтернету	Так, після завантаження моделі	Так, після завантаження моделі	Ні
Коли обирати саме її	Для простих локальних завдань із невисокими вимогами до якості	Для локальної роботи, коли важливі конфіденційність і краща якість	Для складних завдань, коли важливі швидкість, якість і точність

Підсумкове порівняння моделей

Після виконання експериментів було проведено загальне порівняння трьох мовних моделей за показниками швидкості, якості, стабільності формату, вимог до ресурсів, вартості та способу обробки даних.

За результатами оцінювання якості найкращий результат продемонструвала хмарна модель openai/gpt-oss-120b, яка набрала 10 із 12 балів. Локальна модель qwen3:4b отримала 9 балів, тоді як llama3.2:3b - 4 бали.

Під час вимірювання швидкості хмарна модель також показала кращі результати порівняно з llama3.2:3b. Вона мала менший час до появи першого

токена та забезпечувала вищу швидкість формування відповідей для всіх досліджених типів запитів.

Водночас локальні моделі мають переваги, пов'язані з можливістю роботи без постійного підключення до інтернету та обробкою даних без передавання запитів зовнішнім сервісам.

На мою думку, локальну модель доцільно використовувати, коли важлива конфіденційність інформації, автономність або відсутність окремої оплати кожного запиту. Серед досліджених локальних моделей я обрала б qwen3:4b, оскільки вона продемонструвала значно кращу якість відповідей порівняно з llama3.2:3b.

Хмарну модель доцільніше обирати для складніших завдань, коли пріоритетом є якість відповідей та швидкість роботи, а передавання інформації зовнішньому провайдеру допускається умовами використання системи.

Отже, вибір мовної моделі повинен залежати від вимог конкретного завдання, доступних обчислювальних ресурсів, потреби в конфіденційності та очікуваної якості результатів.

Загальний висновок

У ході виконання лабораторної роботи було досліджено особливості розгортання, налаштування та використання локальних і хмарних мовних моделей. Для проведення експериментів використовувалися локальні моделі llama3.2:3b і qwen3:4b, запущені за допомогою Ollama, а також хмарна модель openai/gpt-oss-120b через програмний інтерфейс API.

Було проведено порівняння міркувальних та інструктивних моделей, досліджено вплив системних повідомлень на формування відповідей, а також проаналізовано відмінності між локальним і хмарним способами виконання запитів.

Під час дослідження продуктивності було виміряно час до появи першого токена, загальну тривалість відповіді та швидкість генерації тексту. Встановлено,

що хмарна модель продемонструвала кращі показники швидкості порівняно з локальною llama3.2:3b.

Оцінювання якості відповідей за спеціальною рубрикою показало, що найвищий результат отримала хмарна модель openai/gpt-oss-120b — 10 із 12 балів. Модель qwen3:4b набрала 9 балів, а llama3.2:3b — 4 бали.

Також було досліджено вплив параметрів `temperature` і `top_p` на випадковість генерації. Встановлено, що збільшення цих параметрів може підвищувати різноманітність відповідей, однак не гарантує їхньої правильності. За низької температури модель демонструвала стабільні результати, але могла повторювати фактичні помилки.

У результаті виконання лабораторної роботи було отримано практичні навички налаштування мовних моделей, використання API, проведення експериментальних вимірювань, аналізу відповідей та оцінювання їхньої якості.

Зроблено висновок, що вибір між локальною та хмарною моделлю повинен здійснюватися з урахуванням вимог до швидкості, якості, обчислювальних ресурсів, конфіденційності даних і можливості автономної роботи.